UEF4.3.Object Oriented Programming

Ms SADEG & Ms BOUSBIA
{s_sadeg , n_bousbia}@esi.dz
Ecole Nationale Supérieure d'Informatique (ESI)

Chapter IV

Arrays and Collections

Part 1. Arrays

Arrays

- An array is a data structure that stores a series of **n** values of **same type** (elementary or object).
  - It can be used as a variable, an argument or a method return type

- Each value is accessed with an **integer index**
  - Let "tab" be an array of integers
  - tab[i] is the ith integer in the array
  - In Java i varies from 0 to n-1 (n is the size of the array)
  - Accessing the "n" cell will cause an error!

Declaration and Initialisation

- To define an integer array we can write:
  `int[] a;` or `int a[];`
  - This instruction only declares the variable "a"
- The "new" operator creates the array:
  ```
  a=new int[100];
  int a[]=new int[100];
  ```
- We can also create an array and initialise it simultaneously with its values
  ```
  int[] primeNumbers= {2, 3, 5, 7};
  ```
- It is even possible to initialise an anonymous array:
  ```
  new int[] {11, 13, 17, 19};
  ```
- This syntax is used to reset an array without creating a new variable
  ```
  primeNumbers =new int[] {11, 13, 17, 19};
  ```

Size and Traversal of an Array

- The size of an array can be retrieved using *arrayname.**length***
- To fill or traverse through an array, we can use a loop:

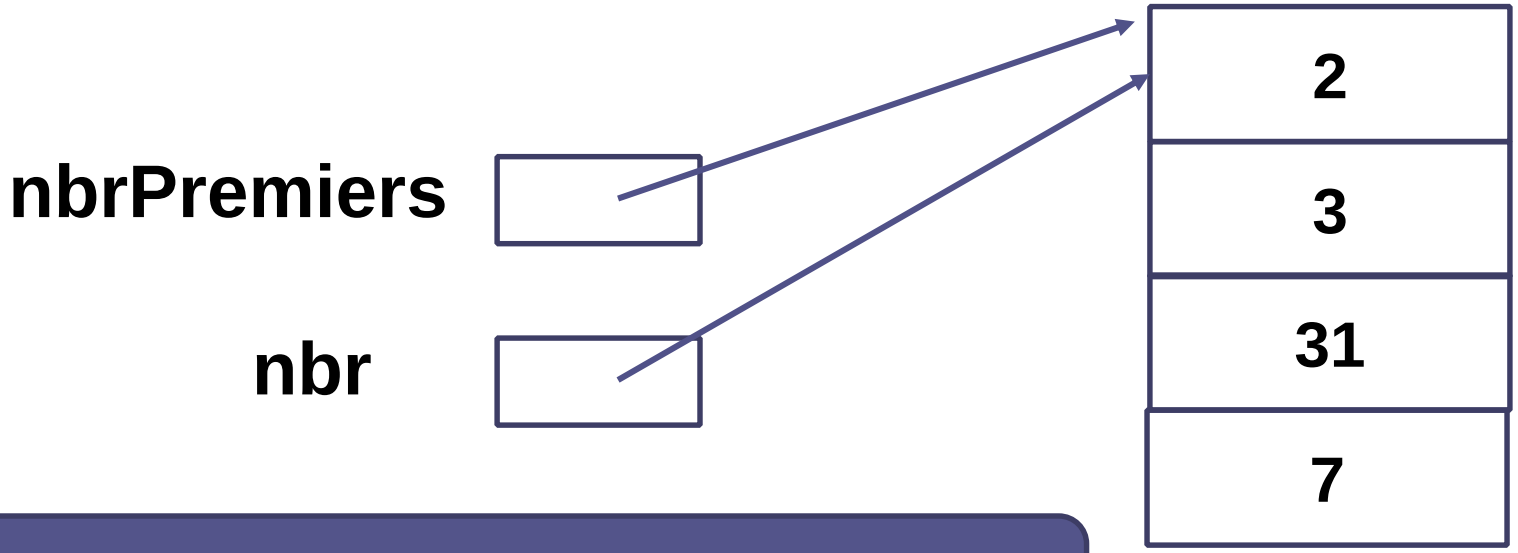
```
for (int i = 0; i < a.length; i++)
      System.out.println(a[i]);
```

- **Note**:
  - It is possible in Java to have an array of size 0, which is different from **null**.
  - This is useful in the case of a method that returns an array as return type

Copying arrays

- What is the result of copying one array variable into another?

```
int[] nbr= primeNumbers;
nbr[2]=31;//primeNumbers[2]= 31
```



Both variables refer to the same array

Copying arrays

One way to copy all the values from one array to another, you should use the arraycopy method of the System class:

```
System.arraycopy(source, sourceIndex, target, targetIndex, length)
```

- Example

```
int[] ta = {2, 3, 5, 7};
int[] tb = {1, 10, 100, 1000, 1010, 1100};
System.arraycopy(ta, 1, tb, 2, 3);
for (int i = 0; i < tb.length; i++)
    System.out.print(tb[i] + " ");
```

- The result

```
1 10 3 5 7 1100
```

Multidimensional Arrays

Multidimensional arrays are arrays of arrays.

They use multiple indices to access the elements.

Example: Suppose we want to manipulate a matrix (a 2-dimensional array):

```
double[][] mat = new double[3][2];
```

Or

```
double[][] mat = {{1, 6}, {5, 8}, {1.5, 7}};
```

Traversal:

```
for (int i = 0; i < mat.length; i++) {
 for (int j = 0; j < mat[i].length; j++) {
        System.out.print(mat[i][j] + " ");}
      }
```

Irregular Arrays (Jagged Arrays)

- A jagged array is a multidimensional array in which the different rows have different lengths (sub-arrays of different sizes).
- To create a jagged array (e.g. triangular matrix), we start by creating the array containing the rows
  - `double[][] mat=new double[3][];`
- Then create the rows of the triangular matrix

```
for (int i = 0; i < mat.length; i++)
    mat[i]= new int[i+1];
```

- Once the array has been allocated, we can fill and access its elements **as long as we don't exceed the limits of each subarray**

Arrays class

- The **java.util package defines an "Arrays" class which** provides static **methods to perform useful operations** such as:

- Searching an array for a specific value to get the index at which it is placed (the binarySearch method).
- Comparing two arrays to determine if they are equal or not (the equals method).
- Filling an array to place a specific value at each index (the fill method).
- Sorting an array into ascending order (the sort method)

Arrays class

operation	*Method*
Search for an element	int binarySearch(char[ ] a) , int binarySearch(int[ ] a) … int binarySearch(Object[ ] a)
Sort the elements of an array	**sort(char[ ] a) , sort(int[ ] a) ….. sort(Object[ ] a) sort(char[ ] a, int fromIndex, int toIndex) , ...**
Fill the table with a value	fill(char[ ] a, char val) , fill(int[ ] a, long val) ….. fill(char[ ] a, int fromIndex, int toIndex, char val)…;
Testing the equality of two arrays	**boolean equals(char[ ] a1, char[ ] a2), boolean equals(int[ ] a1,int[ ] a2), equals(Object[ ] a1,Object[ ] a2)…..**
Representing an array as a String	toString(type[] a)
Copy an array	**copyOf(type[] original, int newLength)**

Arrays class: examples

- Sorting an array: To sort an array, use the static ***sort*** method of the ***Arrays class.***

```
double[] vec = new double[1000];
for (int i = 0; i < vec.length; i++)
        vec[i] = Math.random()*1000;
     Arrays.sort(vec);
```

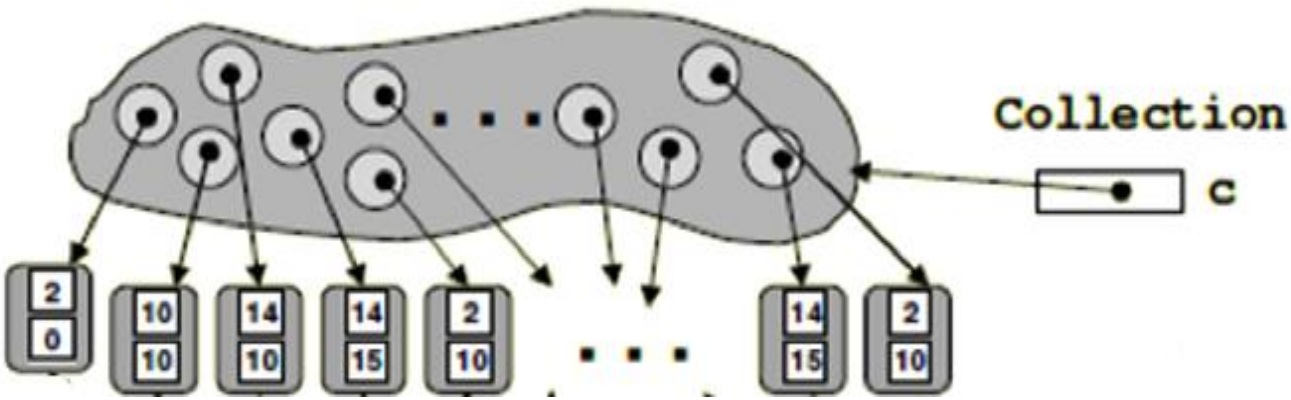
- searching in an array: To search in a **sorted** array, use the static ***binarySearch*** method of the ***Arrays class.*** It returns the index of the ***v*** value found. Otherwise a negative value ***pos***, where (***-pos-1***) indicates the position at which ***v*** should be inserted

```
int pos = Arrays.binarySearch(vec,500);
if (pos >= 0)
System.out.println("position de 500 : " + pos);
else
System.out.println("500 is not in the array, you can
insert it in the box "+ (-pos-1));
```

Chapter IV
Part 2. Collections

Collection

- A *collection* is simply an object that groups multiple elements into a single unit also called a container.
- Collections are used to store, retrieve, manipulate, and communicate aggregate data.



Collections Framework

A *collections framework* is a unified architecture for representing and manipulating collections. It contains

- **Interfaces:** abstract data types that represent collections and allow them to be manipulated independently of the details of their representation.
- **Implementations:** Concrete implementations of the collection interfaces.
- **Algorithms:** methods that perform useful computations, such as searching and sorting, on objects that implement collection interfaces.

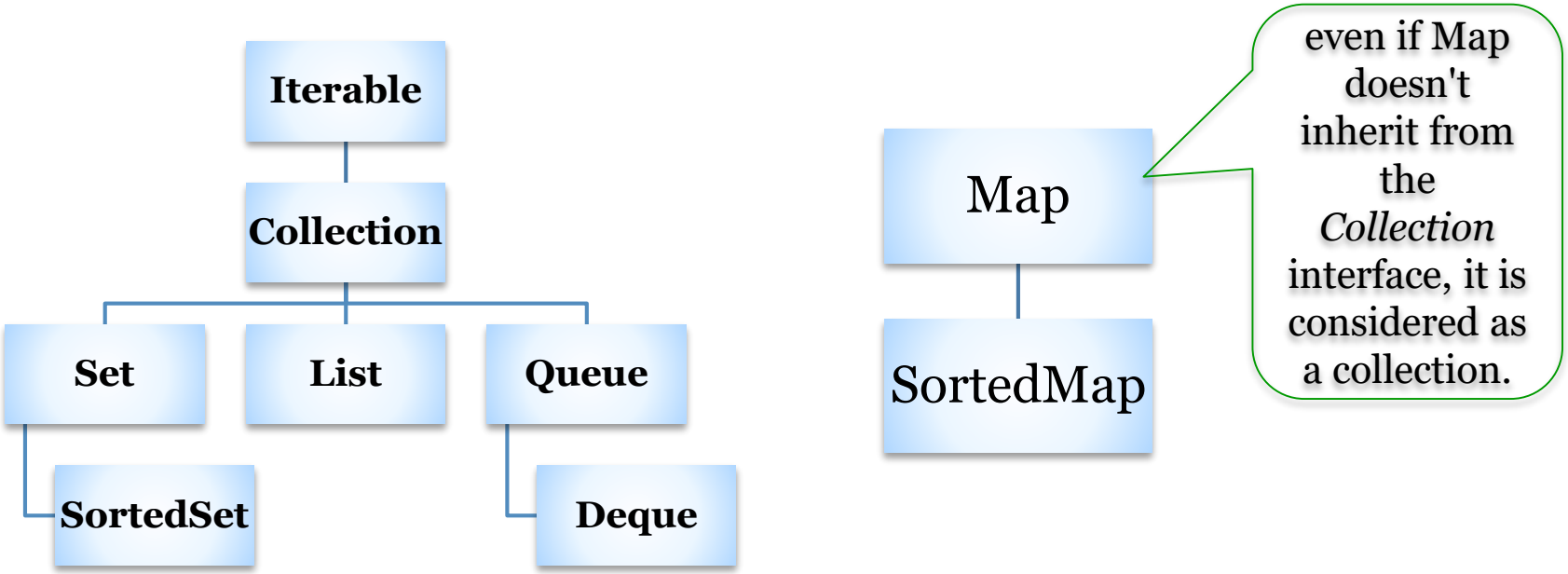
Benefits of the Java Collections Framework

- **Reduces programming effort:** By providing useful data structures and algorithms,and facilitating interoperability among unrelated APIs
- **Increases program speed and quality**: By providing high-performance, high-quality implementations of useful data structures and algorithms.
- **Allows interoperability among unrelated APIs**: Collection interfaces enable different APIs to exchange collections easily, allowing them to work together, even if they were developed separately.
- **Reduces effort to learn and to use new APIs**:Standard collection interfaces eliminate the need to create custom ones
- **Reduces effort to design new APIs**: Designers and implementers can use standard collection interface instead of creating each time an API that relies on collections.
- **Fosters software reuse:** New data structures and algorithms that conform to the standard collection interfaces are by nature reusable.

1. Interfaces: an overview

interfaces

- The *core collection interfaces* encapsulate different types of collections, that allow collections to be manipulated independently of the details of their representation.
- The main interfaces are `Collection` et `Map`



The core collection interfaces.

JCF Interfaces- Collection

- Collection: The root of the hierarchy. It is used to manipulate collections with a maximum degree of generality. The JCF provides no direct implementation of this interface, but does provide implementations for specific sub-interfaces.
  - Set: A collection that cannot contain duplicate elements. It models the mathematical set abstraction
    - SortedSet: A Set that maintains its elements in ascending order.
  - List: A collection that can contain duplicate elements. Its user generally knows the location of the elements and can access them by indicating their positions (integer index) as for an array.
  - Queue: A collection that generally orders its elements according to FIFO order, but each implementation of the queue interface must specify its ordering policy.
    - Deque : this is a double-ended queue that can be used as FIFO or LIFO and is generally used to implement a stack.

JCF Interfaces -Map

- Map: A collection that maps keys to values. A map cannot contain duplicate keys; each key corresponds to at most one value.
  - SortedMap - A map that keeps its elements ordered according to ascending key order. (dictionaries, telephone directory….)

How to use a collection ?

- First, we must choose the structure best suited to the problem

Example: Directory => Map

A set that accepts duplicates => List

- Then, we choose the right implementation

	Set collection of single elements	List collection with duplicates	Map collection in key/value form
Resizable Array		**ArrayList**	
Tree	TreeSet		TreeMap
Linked list		**LinkedList**	
Collection that use a hash table	HashSet		HashMap

Declaring and Initializing Collections

- Declare an object of the chosen collection type:

Example: a redimentional array

```
List list = new ArrayList();
            Or
ArrayList list = new ArrayList();
```

This declaration creates an ArrayList collection whose elements are of type Object.

=> We must cast an element to retrieve it.

Declaring and Initializing Collections

Generics

- Generics enable *types* (classes and interfaces) to be parameters when defining classes, interfaces and methods. It allows the definition of modules that are parameterized by the type they manipulate.
- In the case of collections, this allows interfaces or classes to be declared whose element type is a parameter to be specified.
- All JCF interfaces are generic.
  - Example: the declaration of the Collection interface

    ```
    public interface Collection<E> …
    ```
- The benefits of using generics are:
  - Stronger type checks at compile time
  - Elimination of casts
  - Enabling programmers to implement generic algorithms

Declaring and Initializing Collections

Generics

- Here's an example:

```
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0);
```

Another version of this example using generics

```
List<String> list = new ArrayList<String>();
list.add("hello");
String s = list.get(0);
```

Since JDK7, we can also use the diamond operator

```
List<String> list = new ArrayList<>();
list.add("hello");
String s = list.get(0);
```

The compiler uses the information on the left-hand side to infer the type.

Collections class

- As for arrays, The java.util package defines Collections (with an s at the end) class that provides static methods for collections. Here are some of them

Method	Description
sort(List<T> list)	Sorts the elements of the list in natural order.
reverse(List<?> list)	Reverses the order of elements in the list.
shuffle(List<?> list)	Randomly permutes the elements in the list.
max(Collection<? extends T> coll)	Returns the maximum element according to natural ordering.
min(Collection<? extends T> coll)	Returns the minimum element according to natural ordering.
frequency(Collection<?> c, Object o)	Counts how many times an object appears in a collection.

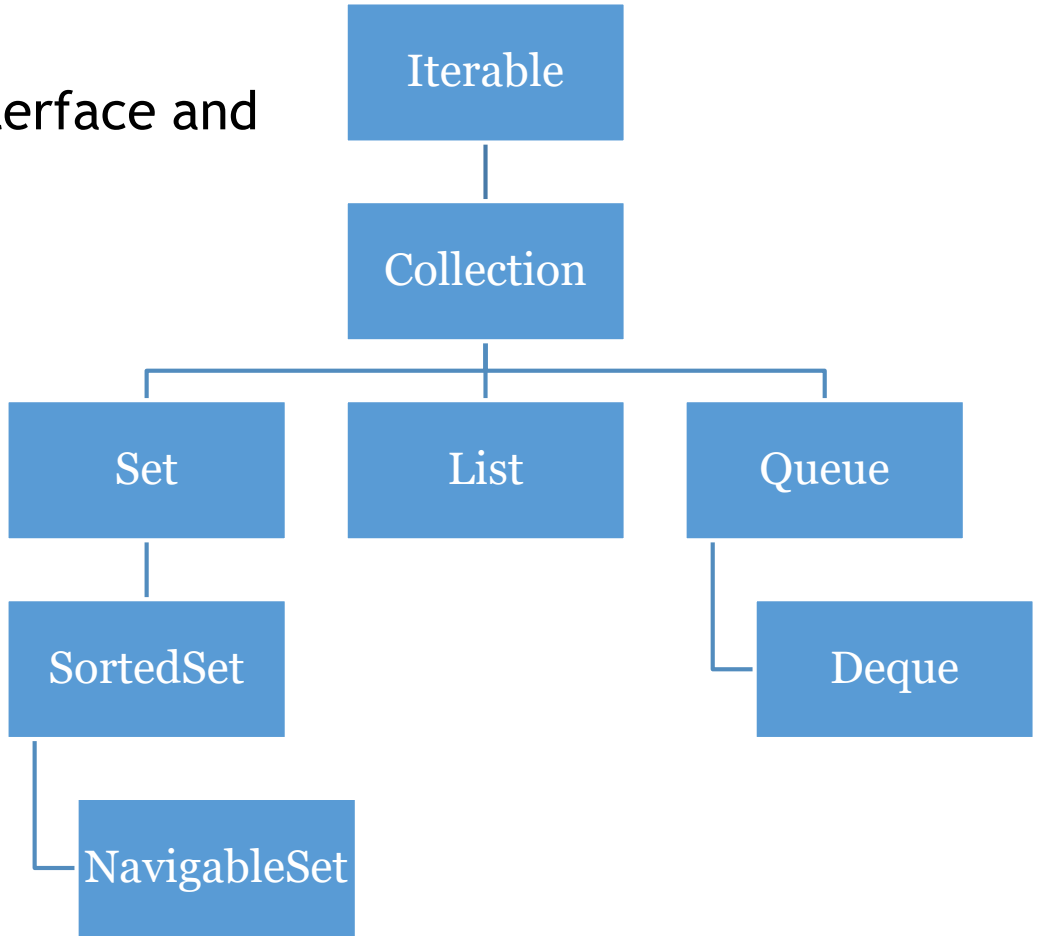
Collection interface

java.util.Collection interface

It's the root of the hierarchy.

It derives from the Iterable interface and its sub-interfaces are:

- List
- Set
- SortedSet
- NavigableSet
- Queue
- Deque



java.util.Collection interface

- The **Collection** interface contains methods that perform basic operations, such as

Method	*Description*
int size	Returns the number of items in the collection
boolean isEmpty()	Returns true if the collection is empty
boolean contains (Object o)	Returns true if the collection contains object o (based on the equals method of class E)
boolean add (E o);	Adds element o to the collection. Returns false if duplicates are forbidden.
boolean remove(Object o)	Deletes object o
Iterator<E> iterator()	Returns the collection iterator

java.util.Collection interface

It also contains *Bulk operations* that perform an operation on an entire Collection, such as

Method	***description***
boolean containsAll(Collection<?> c)	Returns true if the collection contains all the elements of c (the elements of c must be of type E or a subtype of E).
boolean addAll(Collection<? extends E> c)	Add elements of c (union)
boolean removeAll(Collection<?> c)	Deletes all elements of c (difference)
boolean retainAll(Collection<?> c)	Keeps the elements of c (intersection).
void clear ()	Delete all items in the collection

Collections and primitive types

- Java's java.util collections cannot store primitive type values.
- Before JDK 5, it was necessary to explicitly use the wrapper classes for primitive types (e.g., Integer, Double, etc.).
- Starting from JDK 5, conversions between primitive types and their wrapper classes can be done implicitly through **boxing** and **unboxing** (wrapping/unwrapping).

Collections and primitive types

- Example of a **list with** (un)boxing

```
List<Integer> l = new ArrayList<Integer>();
l.add(10);
l.add(-678);
l.add(87);
l.add(7);
int i = l.get(0);
```

- Example of list **without** (un)boxing list (before jdk5)

```
List l = new ArrayList();
l.add(new Integer(10));
l.add(new Integer(-678));
l.add(new Integer(87));
l.add(new Integer(7));
int i =((Integer)l.get(0)).intValue();
```

Traversing Collections

There are three ways to traverse collections:

1. By using Iterators
2. With the for-each construct
3. Using aggregate operations

Some collections (such as lists) offer their own methods.

1. Using an iterator (1/2)

- An iterator is an instance of a class that implements the ***Iterator*** interface.

- It allows you to traverse through a collection and selectively remove elements as needed.

- An iterator for a collection is obtained by calling the iterator method, which is the only method of the **Iterable<E** interface>

- Note:
  - **The "*Collection*" interface derives from the "*Iterable" interface*.**
  - **All collections implement an iterator() method that returns an iterator**

1. Using an iterator (2/2)

- This interface defines methods for objects capable of browsing the data in a collection:

Method	*Description*
boolean hasNext()	Returns true if there is at least one element left to browse in the collection
E next()	Returns the current element in the path and passes the iterator to the next element (throws NoSuchElementException when there's nothing left to return)
void remove()	Deletes the last element browsed (next must be called first)

```
Iterable<E> collection = ...
Iterator<E> it = collection.iterator();
while ( it.hasNext() ){
E o = it.next();
…
}
```

2. Using for-each construct

- It is also possible to use ***for - each*** to browse the collection.

```
Iterable<E> collection = .......;
for (E o : collection) {
  //handle object o
     ...;
}
```

```
Iterable<E> collection = ...
Iterator<E> it = collection.iterator();
while ( it.hasNext() ) {
E o = it.next();
...
}
```

Note: Use `Iterator` instead of the `for-each` construct when you need to remove the current element. The `for-each` construct hides the iterator, so you cannot call `remove`.

Set interface

java.util.Set interace

- A Set is a Collection that models the mathematical set abstraction.
- It cannot contain two objects that are equal according to the equals method (similar to sets in mathematics).

- **Warning!** Non-duplication of objects is not guaranteed if objects already added are modified.

Set interface methods

- The Set interface contains only methods inherited from Collection but methods are adapted to add restriction that duplicate elements are prohibited.
- Set also adds a stronger contract on the behavior of the equals and hashCode operations
- For example, the method **add** does not add an element if an equal element is already in the set (the method then returns **false)**

Interfaces derived from the Set

- SortedSet<E>
  - A **Set** that orders its elements
  - Methods are added to **Set**, related to the total order used to sort the elements (**first(), last(), comparator(), subSet(E debut, E fin)**, **headSet(E fin), tailSet(E debut))**

- NavigableSet<E
  - JDK 6 interface derived from **SortedSet**
  - Allows you to navigate within a **Set** from one of its elements, in the order of the **Set** or in reverse order

Set implementations

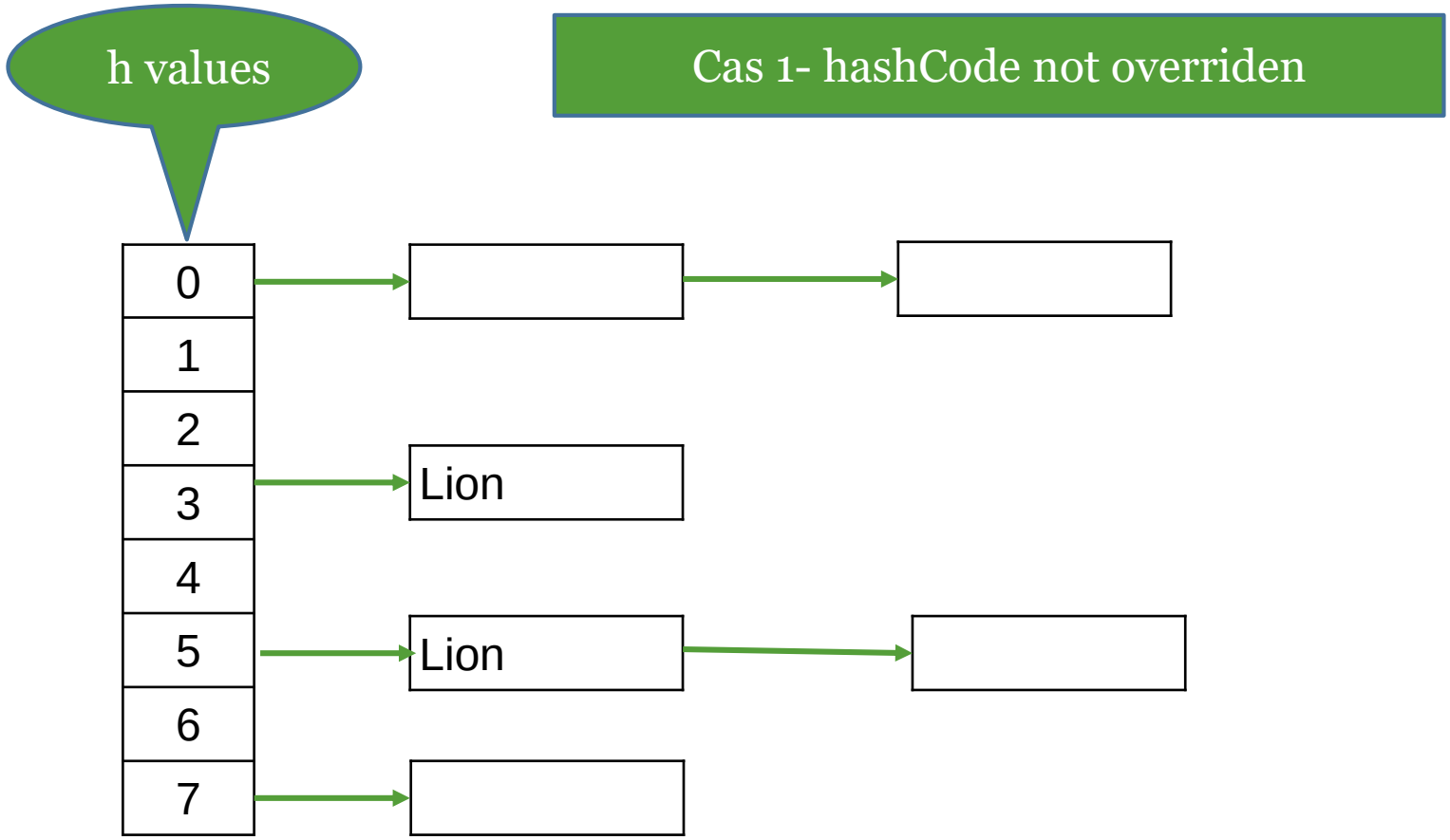
- **HashSet<E>** implements **Set** with a hash table ;
  - This class checks equality only for objects that have the same **hashCode (Object class method)**

  *"2 objects equal in the sense of equals must have the same value for the **hashCode*** method."

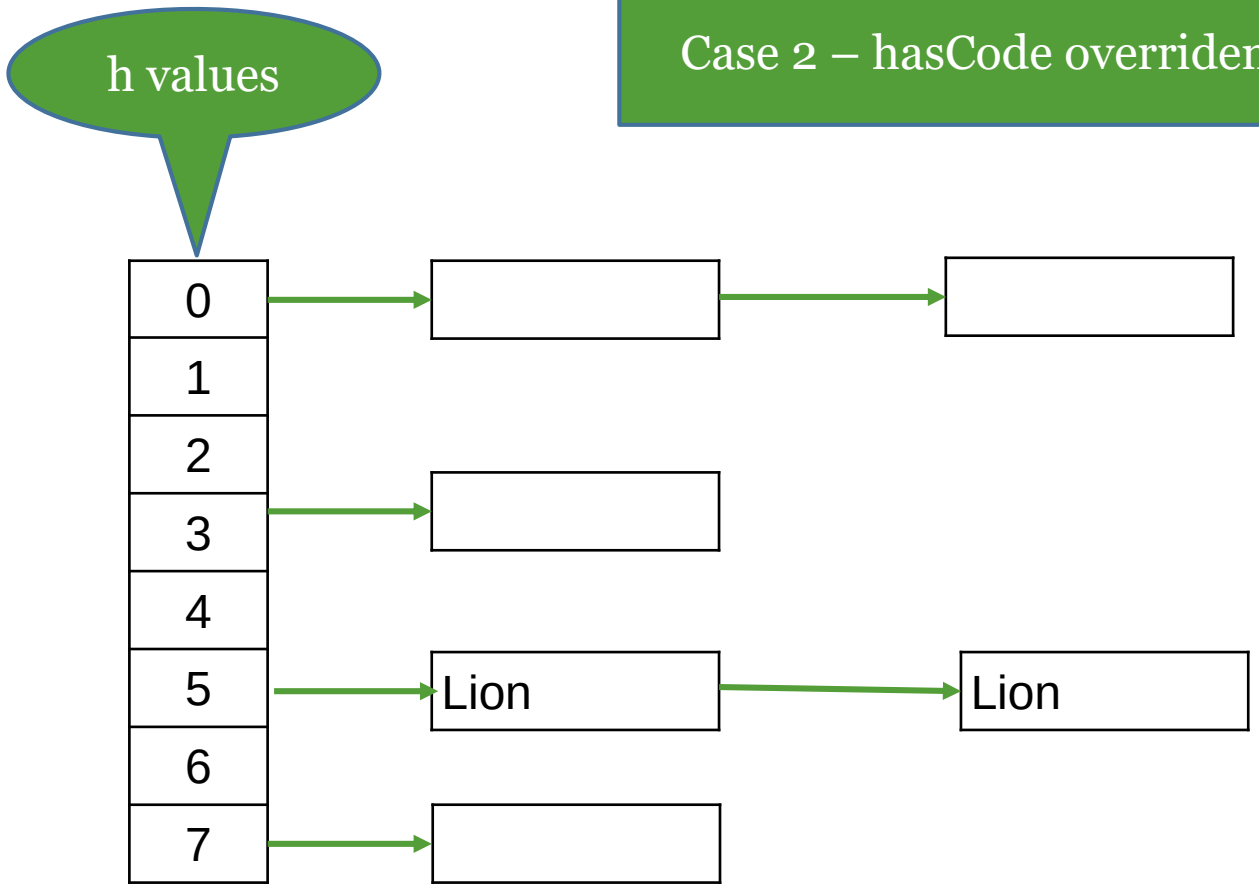
- **TreeSet<E>** implements NavigableSet with an ordered tree
  - stores its objects in a binary tree
  - elements are arranged in their natural order (Comparable<E> interface) or according to the order of a Comparator<E> passed as a constructor parameter

Exercise 1: Part 1 – Sets

1. Create an Animal class (each animal has a name), and a Main class instantiating a set of animals (in the form of a HashSet).
2. Add some animals to the set.
3. Display the animal set using the iterator provided by the Iterator() method.
4. Test your HashSet by introducing animals with the same name.
5. Indicate that two animals are equal if they have the same name by redefining the equals method of the Animal class, overloading it as follows: equals(Animal a) (instead of equals(Object o)).
6. Add several animals with the same name.
7. Display again. What do you notice? Does this correspond to the Set specification?
8. Comment on the equals(Animal a) method and redefine equals(Object o). Does this solve the problem?
9. Redefine the hashCode method of the Animal class so that it returns the hashCode of the animal's name. Does this solve the problem?
10. Transform the set of animals into a TreeSet. What should be added to the Animal class? Why or why not?


h values
Cas 1- hashCode not overriden
0
1
2
3
4
5
6
7
Lion
Lion

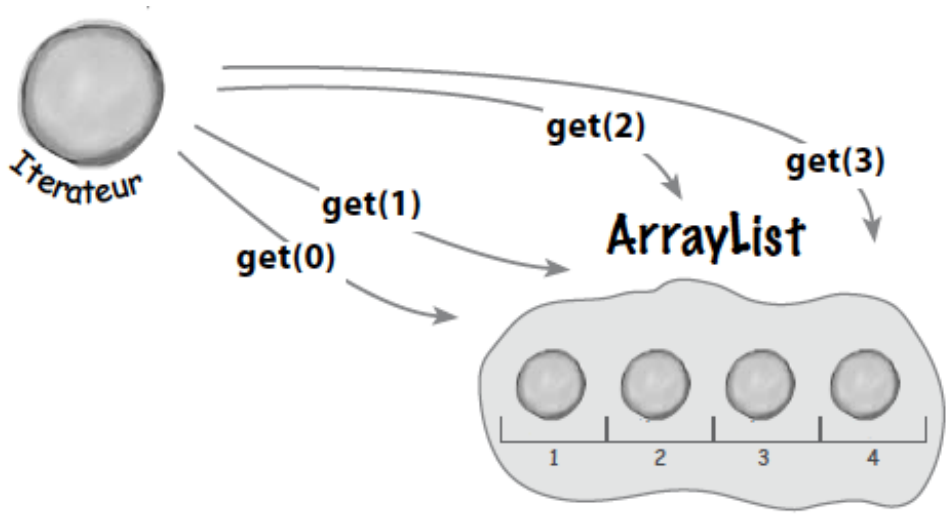
Case 2 – hasCode overriden but not equals



List interface

java.util.List interface

- The `List<E>` interface represents a collection of objects indexed by integers starting at 0.
- Each element can be accessed based on its position in the list.



java.util.List interface

Method	Role
E get(int i)	Returns the element at index i
int indexOf(Object o)	Index of the 1st element equal to 0 (-1 otherwise)
int lastIndexOf(Object o)	Last index of o in the list
List<E> subList(int from, int to)	List of elements [from..to[
ListIterator<E> listIterator()	Two-way iterator
ListIterator<E> listIterator(int i)	Iterator initialized at index i
Optional	
void add(int i, E e)	Add e to index i (insert with right shift)
boolean addAll(int, Collection<? extends E> c)	Insert c from i
E remove(int i)	Delete element at index i (left shift)
E set(int i, E e)	Replace the element at position i with the element e

Iterator for lists: ListIterator<E>

- This interface defines methods for traversing the list in both directions and performing updates that affect on the current element during iteration.
- In addition to the methods defined in the **Iterator interface**, which it extends, the list interface provides the following methods:

Method	Role
void add(E e)	Adds e before the current position of the iterator
boolean hasPrevious()	Returns true if there is at least one element left to traverse in the list in reverse order.
E previous()	Returns the previous element in the list
void set(E e)	Replaces the current element with the one provided as a parameter
int nextIndex()	Returns the index of the next element that will be returned when next is called.
int previousIndex()	Returns the index of the next element that will be returned when calling previous.

Be careful with lists of integers!

- There are **2 remove methods** in the List interface: **remove(int)** and **remove(Object)**

- So what does the code **li.remove(7536) do** if **li** is a **List<Integer>?**
  - Will autoboxing (implicit boxing/unboxing) apply here?
    - No — the 7537th element (at index 7536) will be removed from the list.
    - To remove the actual integer value 7536 from the list, you must ***explicitly*** cast the argument: **li.remove((Integer) 7536);**

List Implementations

- **ArrayList<E>** : A resizable array implementation. Its size increases dynamically as elements are added.
- *Vector<E>* : *An older class introduced before* `ArrayList`*, also designed to provide resizable arrays. It is now largely replaced by* `ArrayList`*.*
- **LinkedList<E>** : Implements a doubly linked list. I
  - t provides several methods for adding, retrieving, or removing elements at the beginning or end of the list, making it suitable for implementing stacks or queues.
  - Key methods include: **addFirst(E e), addLast(E e), getFirst(), getLast(), removeFirst(), and removeLast()..**

- **ArrayList** is generally preferred for most use cases due to its efficient random access. However, if your application involves frequent insertions or deletions in the middle of the list, **LinkedList** may be more appropriate — it avoids shifting elements and implements the **Deque** interface, which will be discussed later.

Exercise 1: Part 2 – List

1. Create a list of animals using the animals from Part 1 (use the appropriate constructor of the ArrayList class).
2. Add several animals with the same name to the list, and add the same Animal instance multiple times. Display the list.
3. Write a static method that takes a Collection of animals as a parameter and displays the names of those whose name starts with "C". To do this, use the iterator provided by any class that implements the Collection interface. Test it with both the set of animals and the list of animals.
4. Sort the list of animals using the static method Collections.sort(List l) (note the s at the end of Collections) and display the result.

Queue interface

java.util.Queue interface

- Inherits from the Collection interface, introduced in Java 1.5.
- It represents a general-purpose collection.
  - It typically follows a FIFO (First-In, First-Out) policy, The "first" element in the queue is called the head of the queue
  - it's the one that will be returned or removed first.
- Elements are added to the tail of the queue.
  - null elements are generally not allowed in a Queue. =>You should not add null elements to a queue.

java.util.Queue interface

- It defines two groups of methods that perform the same basic operations:
  - One group, inherited from Collection, throws an exception when an operation fails (add, remove, element).
  - The other group returns a special value (false or null) under the same conditions (offer, poll, peek).

	Throws an exception	Returns a value
Add(enqueue an element)	add(E);	boolean offer(E e);
Remove (dequeue the head))	E remove();	E poll();
Examine (retrieves a reference to the head of the queue *without* removing it)	E element();	E peek();

Queue Implementations

- **LinkedList<E>** : already covered
- **PriorityQueue<E>** : elements are automatically ordered.
  - The element retrieved from the head of the queue is the *smallest one*.
- Other classes and interfaces are related to concurrency (multi-threading) issues.

- *For more details, refer to the API documentation*.

Example

```
public static void compter(int n) {
Queue<Integer> queue = new LinkedList<Integer>();

//Add elements
for (int i = n; i >= 0; i--)
                queue.add(i);

//remove elements
while (!queue.isEmpty()) {
     System.out.println(queue.remove());
 }
}
```

3
2
1
0

Deque interface

java.util.Deque interface

- Inherits from the Queue interface, introduced in Java 1.6.
- Represents a queue whose elements can be added '***at both ends***' (both the head and tail of the collection)
- Insertion follows a specific order depending on the type of queue.

	Throws an exception	Returns a value
Add(enqueue an element)	boolean addFirst(E e) boolean addLast(E e)	boolean offerFirst (E e) boolean offerLast (E e)
Remove (dequeue the head))	E removeFirst () E removeLast ()	E pollFirst () E pollLast ()
Examine (retrieves a reference to the head of the queue *without* removing it)	E getFirst () E E getLast ()	E peekFirst () E peekLast ()

java.util.Deque interface

- When a **deque** is used as a **queue**, it operates as a **FIFO (First-In-First-Out)** structure, managed with the following methods: addLast(e), offerLast(e), removeFirst(), pollFirst(), getFirst(), peekFirst()
- A **Deque** can be used as a stack (LIFO) using: addFirst(e), removeFirst(), getFirst()
- This interface is implemented by the classes:
  - **LinkedList**<E>
  - **ArrayDeque**<E> (the most commonly used) : This is the recommended class for implementing a LIFO (whereas java.util.Stack is not recommended for the same reasons as Vector).

Example

```
import java.util.*;

public class ListExample {
public static void main(String args[]) {
Deque queue = new LinkedList<String>();
queue.addFirst("Green");
queue.addFirst("Blue");
queue.addFirst("Red");
System.out.println(queue);
queue.removeFirst();
System.out.println(queue);
queue.removeFirst();
System.out.println(queue);
}
}
```

Result:

[Red, Blue, Green]
[Blue, Green]
[Green]

Map interface

Common problem

- In computer science, we often need to retrieve information using a unique identifier key.
  For example:
  - Looking up a phone number when you know a person's name
  - Searching for student details when you have their ID number

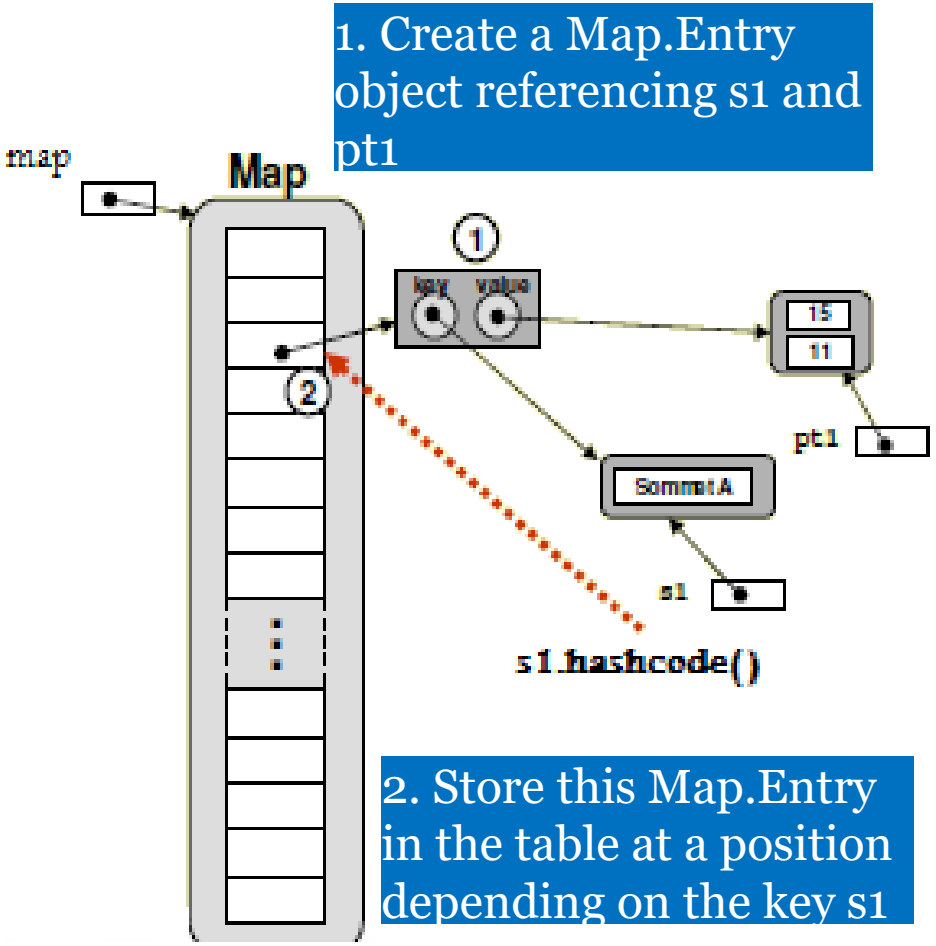
→ To solve this problem, we typically use associative arrays, implemented in Java as the Map interface.

java.util.Map interface

- The **Map<K,V>** interface represents a collection of key-value pairs (where **K** is the key type and **V** the value type).
- Key properties:
  - Each key uniquely identifies exactly one value.
  - A map cannot contain duplicate keys (two keys k1 and k2 cannot satisfy k1.equals(k2)).
- Common operations include:
  - Adding/removing key-value pairs (also called entries).
  - Retrieving a value by its key.
  - Checking if the map contains a specific key.
  - Checking if the map contains a specific value.

Map interface : Example

```
Map<String, Point> map=
  new HashMap<String,Point()>;
String s1=new String("Sommet A");
Point pt1=new Point (15,11);
map.put(s1,pt1);
```



java.util.Map interface

Method	Role
V **put**(K k, V v)	Add v with key k: returns the previous value associated with the key if it already existed
V **get**(Object k)	Returns the value associated with key k, or null
V r**emove**(Object k)	Remove the entry associated with k
boolean **containsKey**(Object k)	Checks if key k exists in the map - returns true if found
boolean **containsValue**(Object v)	Returns true if value v exists in the map
void **putAll**(Map<? extends K,? extends V> m)	Adds all entries from map m
int **size**()	Returns the number of entries in the map
boolean **isEmpty**()	Returns true if the map is empty
void **clear**()	Empties the map

Internal interface Entry<K,V> of Map interface

- The Map<K,V> interface contains an inner public interface Map.Entry<K,V> that represents a key-value pair.
- This entry interface provides three core methods:
  - K getKey()
  - V getValue()
  - V setValue(V value)
- The Map's entrySet() method returns a Set<Map.Entry<K,V>> object (a set of all entries in the map).

Retrieve data from a Map

- Set<Map.Entry<K, V>> **entrySet()**
  - Retrieves the entries (key-value pairs) as a Set<Map.Entry<K,V>>
- Collection<V> **values()**
  - Retrieves the values as a Collection<V>
- Set<K> **keySet()**
  - Retrieves the keys as a Set<K>

- We use the iterator() method from the appropriate Collection interface (Collection or Set) to retrieve values one by one

Iterating Through Maps

- Iterate through Map keys

```
Map<Key, Value> map=…;
for (Key key : map.keySet())
{
// manipulate the key...
…
}
```

Iterating Through Maps

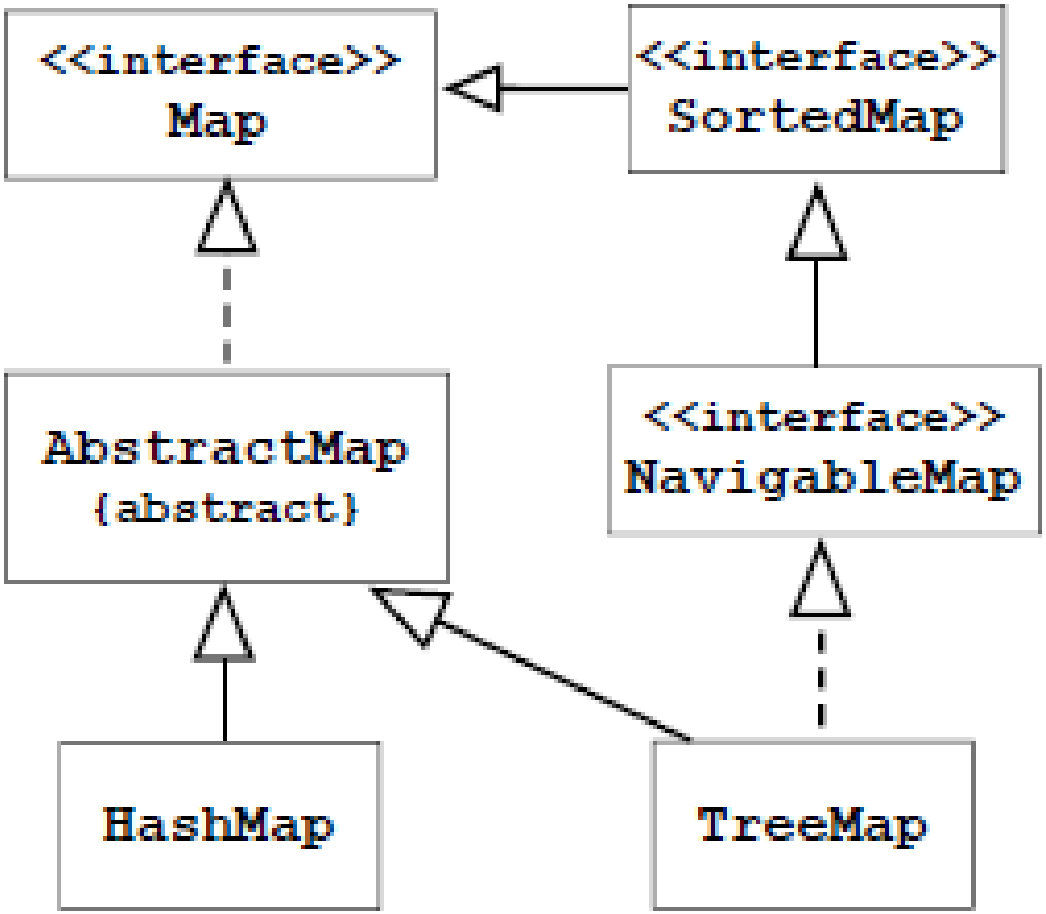
- Iterating Through All Keys and Values in a Map

```
for (Key key : map.keySet()) {
  Value value = map.get(key); // l'appel de get
engendre un temps coûteux en accès à chaque itération
 }
```

Or

```
for (Map.Entry<Key, Value> entry : map.entrySet())
  {
   Key key = entry.getKey();
   Value value = entry.getValue();
  // manipuler key et value...
  }
```

Map Implementations



Map Sub-Interfaces

- **SortedMap<K,V> interface**
  - This is a Map that maintains total ordering of its keys (using either natural ordering of K or a comparator provided with the Map).
  - It extends Map with methods to extract sub-maps and access the first/last keys (similar to SortedSet functionality).

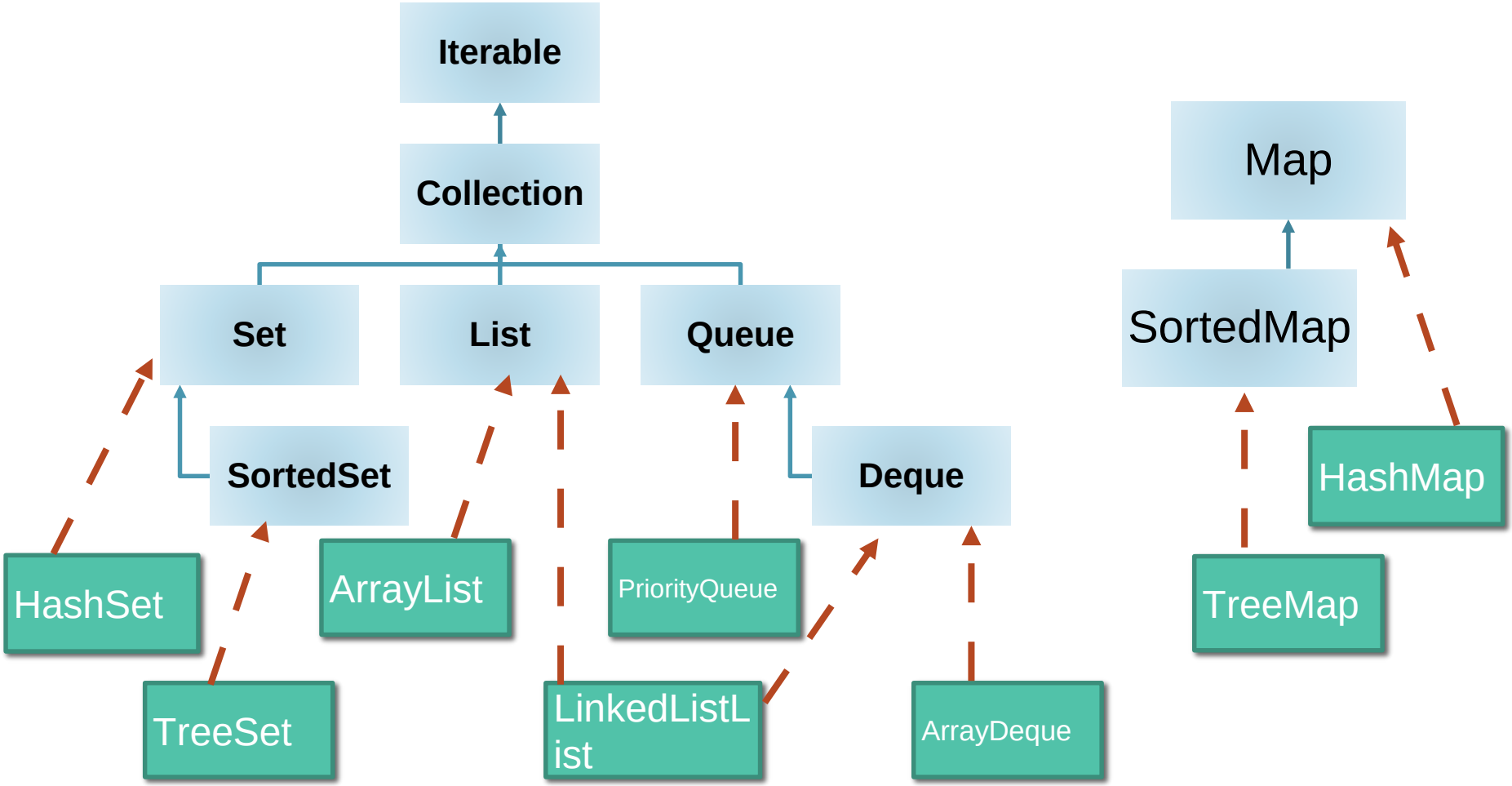
- **NavigableMap<K,V> interface**
  - Extends SortedMap with navigation capabilities
  - Allows traversing the map starting from any given key, in either the set's natural ordering, or reverse order

- *For more details, refer to the API documentation*.

Map Implementations

- **HashMap<K,V>**
  - A data structure that allows extremely fast object retrieval when the key is known (constant-time access).
  - Internally, it uses an array-based storage system and a hash function applied to keys
  - The hashCode() method (inherited from Object or overridden) serves as the hash function.
- **TreeMap<K,V>**
  - An ordered tree structure sorted by **key values**.
  - Implements NavigableMap<K,V> with ordering determined by either:
    - Natural ordering (via Comparable<? super K>), or
    - A custom Comparator<? super K> provided to the constructor.

JCF Principal implementations



Exercise 1: Part 3 – Map

1. We associate each animal with a tattoo composed of its name and a unique integer. Write the corresponding Tattoo class. The unique integer can be generated via a static integer counter.
2. Create a Map of animals containing animals indexed by their tattoo (you may choose the Map implementation class: TreeMap, HashMap, etc.).
3. Display all entries of your Map (use the entrySet() method).
4. Display all keys and their associated values from your Map.